

CAPSTONE PROJECT ASSIGNMENT SHEET

Project Overheat: Predictive Maintenance AI

1. Project Overview

At **AeroLogistics**, our global fulfillment centers rely on high-speed automated sorting robots. Recently, the primary drive motors in these robots have been unexpectedly overheating and failing, causing catastrophic warehouse shutdowns that cost \$50,000 per minute in delayed shipments.

As a Data Science intern, your objective is to transition our maintenance floor from a reactive strategy to a proactive one. You must ingest Internet of Things (IoT) sensor telemetry, engineer time-series features to capture mechanical degradation, and build a Machine Learning model capable of predicting a mechanical failure at least 24 hours before it actually occurs.

2. Data Architecture & Core Challenges

You have been provided with one continuous time-series data asset:

- `iot_sensor_telemetry.csv`: 10,000 hours of synchronous machine logs capturing `Sensor_Temp_C`, `Sensor_Vibration_mm`, `Sensor_Voltage_V`, and a Boolean `Catastrophic_Failure` indicator.

To complete this assignment successfully, your codebase must navigate four production hurdles:

1. **The Windowing Trap (Feature Engineering):** A single anomalous mechanical jolt does not indicate failure; sustained shaking does. You must engineer Rolling Means and Rolling Standard Deviations across multi-hour windows to capture degrading mechanical health over time.
2. **The Lead-Time Trap (Target Shifting):** If you train your model to predict the exact moment of failure, the AI will simply state, *"The machine is breaking right now,"* which is useless to a mechanic. You must dynamically shift the target variable backward so the model learns to predict failures exactly 24 hours in advance.
3. **The Data Leakage Trap:** Standard machine learning algorithms randomly shuffle rows during train/test splits. Because this is time-series data, shuffling will cause your model to learn from the future to predict the past. You must implement a strict chronological partition.
4. **Explainable AI (XAI):** Mechanics will not replace a \$10,000 motor blindly. You must extract mathematical Feature Importances to prove whether temperature, voltage, or vibration is the leading indicator of the impending failure.

3. Phase-by-Phase Milestones

Your implementation must progress linearly through the following operational phases within your standalone runtime script:

Phase 1: Feature Engineering (Rolling Windows)

Parse the timestamp column and ensure the dataset is sorted chronologically.

Compute a 12-hour rolling mathematical mean for Temperature.

Compute a 12-hour rolling standard deviation for Vibration to capture erratic shaking.

Drop any initial NaN rows generated by the rolling calculations.

Phase 2: Target Shifting

Utilize the Pandas `.shift()` method to move the `Catastrophic_Failure` label up by 24 rows, creating a new `Failure_In_24_Hours` target variable.

Drop the final 24 rows of the dataset (which will now contain null targets) and drop the original failure column to prevent target leakage.

Phase 3: Chronological Train/Test Partition

Isolate your engineered features (`X`) and your shifted target variable (`y`).

Manually slice the dataset: allocate the first 80% of the timeline for model training, and reserve the final 20% for testing. **Do not use random shuffling.**

Phase 4: Modeling & Explainability

Initialize a Random Forest Classifier, ensuring you apply balanced class weights to penalize the model for missing rare failure events.

Extract the `feature_importances_` array from the trained model.

Generate a Matplotlib horizontal bar chart ranking the engineered features by their predictive power, and write a brief business recommendation for the maintenance crew.

4. Final Submission Standards & Evaluation

Deliverable Constraint: You must submit exactly **one** file: `predictive_maintenance_final.ipynb`. This Jupyter Notebook will be run from a clean state by the evaluation team and must execute sequentially without runtime failures. It will be graded out of 100 points across the following criteria:

- **Time-Series Engineering (25%)** — Mathematical accuracy of rolling windows and NaN handling.
- **Target Shifting & Leakage Prevention (25%)** — Correct implementation of the 24-hour lead time and chronological data splitting.
- **Model Optimization (25%)** — Handling class imbalance and achieving high recall for failure events.

- **Explainable AI & Visualization (25%)** — Quality of the Matplotlib charts and the clarity of the final business recommendation.